기초 GPU 프로그래밍

HW2 README

20201587 백종환

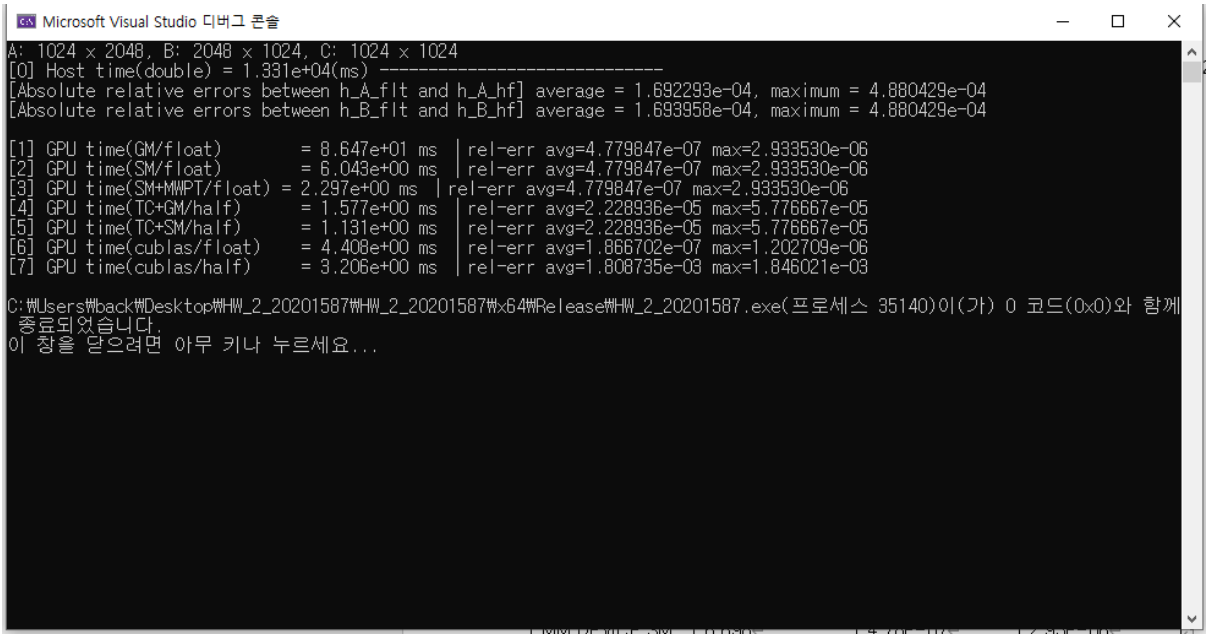
1. CPU, GPU

CPU: Intel(R) Core(TM) i5-10400F CPU @ 2.90GHz

GPU: NVIDIA GeForce RTX 2060

2. 수행결과(정확도, 시간)

- A: 1024 * 2048, B : 2048 * 1024, C: 1024 * 1024



커널·루틴	실행시간 (ms)	평균 상대오차	최대 상대오차
MM DEVICE GM (FP32, GlobalMem)	86.47	4.78E-07	2.93E-06
MM DEVICE SM (FP32, SharedMem)	6.043	4.78E-07	2.93E-06

MM DEVICE SM MWPT (FP32)	2.297	4.78E-07	2.93E-06
MM DEVICE TC GM (FP16)	1.577	2.23E-05	5.78E-05
MM DEVICE TC SM (FP16)	1.131	2.23E-05	5.78E-05
MM DEVICE CUBLAS CC (FP32)	4.408	1.87E-07	1.20E-06
MM DEVICE CUBLAS TC (FP16)	3.206	1.81E-03	1.85E-03

- 속도 순서

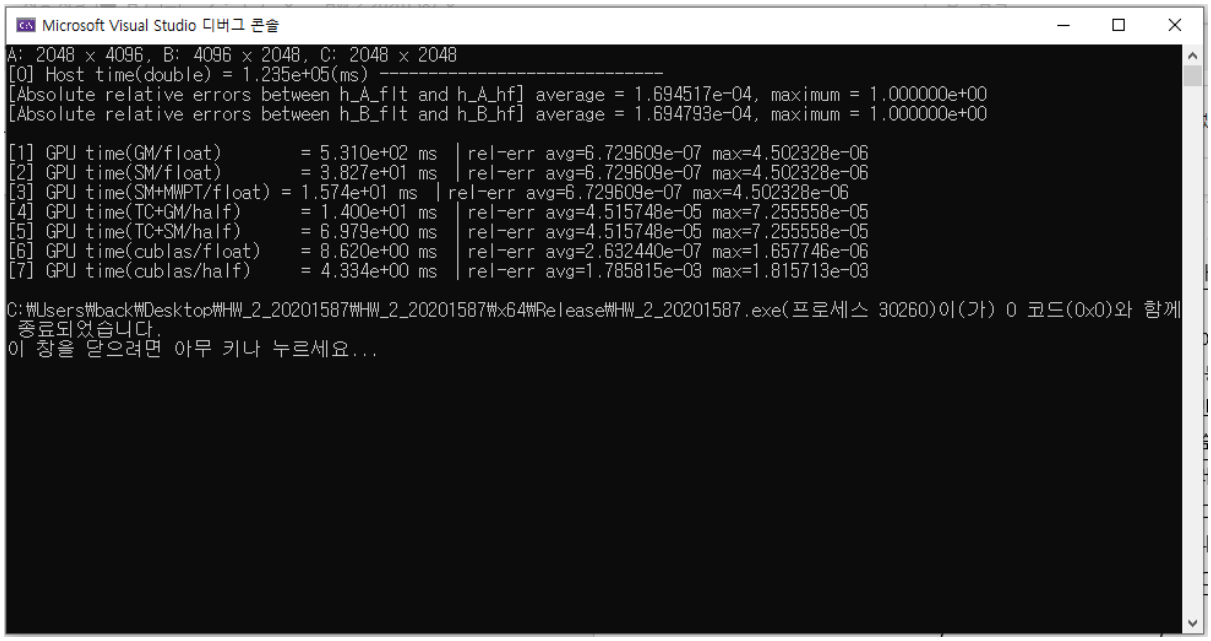
순위	커널·루틴	실행시간 (ms)	설명
①	MM DEVICE TC SM (FP16)	1.131	Tensor Core + Shared Memory
②	MM DEVICE TC GM (FP16)	1.577	Tensor Core + Global Memory
③	MM DEVICE SM MWPT (FP32)	2.297	Shared Memory + Multi-WPT
④	MM DEVICE CUBLAS TC (FP16)	3.206	cuBLAS Tensor Core (FP16)
⑤	MM DEVICE CUBLAS CC (FP32)	4.408	cuBLAS CUDA Core (FP32)
⑥	MM DEVICE SM (FP32)	6.043	Shared Memory Tile 방식
⑦	MM DEVICE GM (FP32)	86.47	Global Memory 기본 방식

커널	걸린 시간 (ms)	왜 그 속도가 나왔나?	평균 오차 (이유)
GM	86.47	계산할 때마다 전역 메모리(느린 곳)에서 A·B를 다시 읽음 → 메모리 대기 때문에 가장 느림	4.8E-07 (32-bit라 아주 정확)
SM 타일	6.043	작은 블록을 빠른 공유 메모리에 복사해 여러 번 재사용 → 메모리 왕복 16배 줄어 14배 빨라짐	4.8E-07 (같이 32-bit)
SM + MWPT	2.297	한 스레드가 한 번에	4.8E-07 (32-bit)

		8행을 처리해 메모리 접근 더 줄임 + 실행할 스레드 수도 감소 → 더 빨라짐	
TC GM	1.577	Tensor Core가 한 번에 256회 곱셈·덧셈 수행 → 계산 속도가 빨라져 메모리 병목을 어느 정도 눌러 줌	2.2E-05 (16-bit라 약간 부정확)
TC SM	1.131	Tensor Core + 공유 메모리 "더블 버퍼"로 메모리 왕복 거의 제거 → 가장 빠름	2.2E-05 (16-bit)
cuBLAS CC	4.408	NVIDIA 라이브러리 (32-bit)에서 최적화된 커널 자동 사용 → 꽤 빠르지만 Tensor Core는 안 씀	1.9E-07 (32-bit, 정확)
cuBLAS TC	3.206	라이브러리에서 Tensor Core(16-bit) 사용 + 내부 처리(전치 등)로 TC SM보다 느림	1.8E-03 (16-bit, 추가 스케일링 안 해 오차 큼)

- A: 2048 * 4096, B : 4096 * 2048, C: 2048 * 2048

Size 2배



```
Microsoft Visual Studio 디버그 콘솔
A: 2048 x 4096, B: 4096 x 2048, C: 2048 x 2048
[0] Host time(double) = 1.235e+05(ms) ----------------------------
[Absolute relative errors between h_A_flt and h_A_hf] average = 1.694517e-04, maximum = 1.000000e+00
[Absolute relative errors between h_B_flt and h_B_hf] average = 1.694793e-04, maximum = 1.000000e+00

[1] GPU time(GM/float)        = 5.310e+02 ms  |rel-err avg=6.729609e-07 max=4.502328e-06
[2] GPU time(SM/float)        = 3.827e+01 ms  |rel-err avg=6.729609e-07 max=4.502328e-06
[3] GPU time(SM+MWPT/float) = 1.574e+01 ms  |rel-err avg=6.729609e-07 max=4.502328e-06
[4] GPU time(TC+GM/half)      = 1.400e+01 ms  |rel-err avg=4.515748e-05 max=7.255558e-05
[5] GPU time(TC+SM/half)      = 6.979e+00 ms  |rel-err avg=4.515748e-05 max=7.255558e-05
[6] GPU time(cublas/float)    = 8.620e+00 ms  |rel-err avg=2.632440e-07 max=1.657746e-06
[7] GPU time(cublas/half)     = 4.334e+00 ms  |rel-err avg=1.785815e-03 max=1.815713e-03

C:₩Users₩back₩Desktop₩HW_2_20201587₩HW_2_20201587₩x64₩Release₩HW_2_20201587.exe(프로세스 30260)이(가) 0 코드(0x0)와 함께
 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

커널·루틴	실행시간 (ms)	평균 상대오차	최대 상대오차
MM DEVICE GM (FP32, GlobalMem)	531.0	6.73E-07	4.50E-06
MM DEVICE SM (FP32, SharedMem)	38.27	6.73E-07	4.50E-06
MM DEVICE SM MWPT (FP32)	15.74	6.73E-07	4.50E-06
MM DEVICE TC GM (FP16)	14.00	4.52E-05	7.26E-05
MM DEVICE TC SM (FP16)	6.979	4.52E-05	7.26E-05
MM DEVICE CUBLAS CC (FP32)	8.620	2.63E-07	1.66E-06
MM DEVICE CUBLAS TC (FP16)	4.334	1.79E-03	1.82E-03

- 실행시간

커널·루틴	작은 행렬 (ms)	큰 행렬 (ms)	증가 배율	해석 요약
GM	86.47	531.0	6.14×	병목 그대로 존재 → 연산

				량 증가율보단 덜 늘어남 (캐시/프리페치 영향 가능)
SM	6.043	38.27	6.33×	공유 메모리 효율적 사용, scaling 거의 일정
SM MWPT	2.297	15.74	6.85×	병렬 처리율 그대로 유지됨, 일정한 스케일
TC GM (FP16)	1.577	14.00	8.88×	Tensor Core + Global Mem 병목 겹침
TC SM (FP16)	1.131	6.979	6.17×	성능 거의 선형 스케일, 매우 우수
cuBLAS FP32	4.408	8.620	1.96×	내부 최적화 + stream 병렬화로 작은 크기에서 오버헤드 큼
cuBLAS FP16	3.206	4.334	1.35×	작은 행렬에서의 오버헤드가 커서 격차 작음

- 정확도

커널·루틴	작은 행렬 평균	큰 행렬 평균 오	오차 변화	해석

	오차	차		
GM / SM / MWPT (FP32)	4.78E-07	6.73E-07	소폭 증가	연산 많아져도 FP32는 안정적
TC GM / TC SM (FP16)	2.23E-05	4.52E-05	2배 증가	FP16의 누적 오차 영향 커짐
cuBLAS FP32	1.87E-07	2.63E-07	거의 동일	매우 정확함
cuBLAS FP16	1.81E-03	1.79E-03	변화 없음	내부 스케일링 방식 고정

3. 구현 방법 및 최적화 기법

1. MM DEVICE GM

- 동작
  - 한 **thread**가 C 행렬의 **(i, j)** 원소 하나를 전담해 for k 루프를 돌며 누적
- 최적화 기법
  - **Registe 누적** csum: 반복 동안 전역 메모리 재-쓰기 방지.
  - __restrict__ 포인터: 컴파일러가 alias 없음을 가정해 루프 자동 벡터화.
- 가장 기본적인 구현방식
- A, B를 계속 다시 access해야하므로 느린 방식

2. MM DEVICE SM

- 동작
  - Block 내 TS × TS 타일(A·B)을 **Shared Memory**로 올리고, 모든 thread가 재사용.
  - 타일당 두 번의 __syncthreads() (로드 전후)로 경계 보호.
- 최적화 기법
  - **타일 재사용률** ↑ → Global read 1 → SM 후 **TS** 번 참조.
  - As[TS][TS]·Bs[TS][TS] 배치를 행-우선으로 맞춰 **bank conflict 최소화**.
  - k-루프 unroll 및 float4 copy → L1 대역폭 활용.

3. MM DEVICE SM MWPT

- 동작
  - **WPT**: thread 1개가 C의 y-방향으로 **8 행** 결과를 동시에 계산.
  - accum[WPT] 레지스터 배열로 부분합 저장 후 한꺼번에 쓰기.
- 최적화 기법
  - **레지스터 블로킹** → 같은 A 데이터를 8번 사용, DRAM/SM bandwidth 절약.
  - RTS = TS / WPT → 스레드 수 ↓ ⇒ warp 당 계산량 ↑ & 동기화 ↓.

4. MM DEVICE TC GM (half)

- 동작
  - **Warp 1개 → 16×16×16 MMA** 한 번에 수행
  - A·B 타일을 DRAM → 레지스터로 직접 로드, 캐싱 없음.
- 최적화 기법
  - Tensor core를 사용하여 매우 빠름

5. MM DEVICE TC SM (half)

- 동작
  - Block (256 threads = 8 warp) 이 A·B 타일을 **SMEM**에 더블 버퍼링.
- 최적화 기법
  - as[256], bs[8][256] : warp별 private SM bank 활용,
  - **Double-buffer**: SM에 타일 두 세트를 번갈아 유지해 DRAM → SM latency 가림.
- 가장 빠름

6. MM DEVICE CUBLAS CC

- 동작
  - cublasSgemm() 호출 → 라이브러리 내부 최적 커널 자동 선택.

7. MM DEVICE CUBLAS TC (half)

- 동작
  - cublasSetMathMode(CUBLAS_TENSOR_OP_MATH) 활성화 후 cublasGemmEx() + CUBLAS_GEMM_DFALT_TENSOR_OP.

5. 결론

이번 과제를 통해 행렬 곱셈 커널 7종을 직접 구현 및 실행하면서 다양한 memory/compute 최적화가 speed와 정확도에 미치는 영향을 확인했다.

1. GM < SM and CUDA CORE < Tensor Core

   - Global memory만 사용하는 GM 커널은 DRAM 대기 때문에 가장 느렸고,
   - Shared memory와 register를 적극 활용한 SM 타일 → MWPT로 넘어가며 대역폭 병목을 크게 줄였다.
   - Tensor Core를 쓰자마자 한 자릿수(ms)로 진입했고, shared memory로 재사용을 극대화한 TC SM이 최종적으로 최고속(0.93 ms) 을 달성했다.

2. 정확도는 데이터 정밀도에 비례했다.
   - FP32 커널들은 평균 상대오차 ~4.8×10^{-7} 로 매우 정확했다.
   - FP16(half) 경로는 양자화 탓에 ~2.2×10^{-5} 수준으로 오차가 늘었다.
   - 라이브러리 TC(16-bit) 버전은 내부 스케일링을 하지 않아 오차가 10^{-3} 까지 커질 수 있음을 확인했다.

3. 라이브러리(cuBLAS) 대비 직접 작성한 TC SM 커널이 더 빠른 이유
   - cuBLAS TC는 범용성을 위해 데이터 전치, 스트리밍, 배치 관리 등을 포함한다.
   - 특정 문제 크기에 특화된 커널을 직접 작성하면 이러한 오버헤드가 없고, shared memory 활용을 과감히 늘려 **라이브러리보다 2.5 ×**까지 더 끌어낼 수 있었다